

Project 3: Sparse Neural Network Training and Optimization

Required: 2-3 people

Modern neural networks are often highly over-parameterized, containing millions of weights. However, many of these parameters are redundant, and recent research has shown that neural networks can often be trained effectively even when most connections are removed. Instead of first training a dense network and then pruning it, a newer paradigm is to **train sparse neural networks directly from scratch**. In such models, only a small subset of weights are active during training, which can greatly reduce memory usage and computation.

In this project, students will study **sparse neural network training** and investigate how different optimization strategies behave under extreme sparsity. Let W denote the weight matrix of a neural network. Sparse training introduces a binary mask M such that

$$W = M \odot \theta,$$

where $\odot$ denotes elementwise multiplication and θ are the underlying trainable parameters. Entries where $M_{ij} = 0$ correspond to inactive connections that remain zero throughout training.

Students will train sparse convolutional neural networks on the CIFAR-10 dataset and explore three increasingly sophisticated approaches: static sparse training, Sparse Evolutionary Training (SET), and RigL.

Dataset and Model

The experiments will be conducted on the **CIFAR-10** image classification dataset. Students should train a small convolutional neural network such as **ResNet-20** (or a comparable lightweight ResNet architecture). To study the effect of sparsity, models should be trained at several sparsity levels, for example:

- **90% sparsity** (10% active weights)
- **95% sparsity**
- **98% sparsity**

The goal is to understand how network performance changes as the number of active parameters decreases.

Task 1: Sparse Networks from Scratch

In the first task, students will train sparse networks with a **fixed random connectivity mask**. The mask M is generated at initialization and remains unchanged throughout training. Students should compare at least two optimization methods: **SGD with momentum** and **Adam**. The goal is to investigate how different optimizers behave when training highly sparse networks. Suggested questions include:

- How does sparsity affect convergence speed?
- How does accuracy degrade as sparsity increases?
- Do different optimizers behave differently under high sparsity?

Task 2: Sparse Evolutionary Training (SET)

In the second task, students will implement **Sparse Evolutionary Training (SET)** [\[Mocanu et al. 2018\]](#), which allows the sparse connectivity pattern to evolve during training. At regular intervals during training:

- Remove a fraction of the smallest magnitude weights.
- Add the same number of new connections at random locations.

This procedure maintains a constant sparsity level while enabling the network structure to adapt over time. Students should compare the performance of SET with the static sparse baseline.

Task 3: Rigging and Lottery (RigL)

In the third task, students will implement a simplified version of **Rigging the Lottery (RigL)** (Evci et al., 2020), which improves dynamic sparse training by using gradient information to guide the growth of new connections. During topology updates:

- Remove weights with the smallest magnitude.
- Add new connections at locations with the largest gradient magnitude.

The intuition is that connections with large gradients would most effectively reduce the loss if they were active. Students should compare RigL with both static sparse training and SET.

Experiments and Analysis

Students should conduct systematic experiments to understand how sparsity affects optimization and model performance. Suggested experiments include:

- **Accuracy vs. sparsity:** Evaluate model performance at multiple sparsity levels.
- **Optimizer comparison:** Compare the training behavior of SGD and Adam under sparse training.
- **Static vs. dynamic sparsity:** Compare fixed sparse training, SET, and RigL.
- **Visualization of convolution kernels:** Visualize learned convolution filters to examine how sparsity is distributed across kernels and layers.
- **Connectivity analysis:** Analyze how sparse connectivity evolves during training in SET or RigL.

Students are encouraged to produce clear plots such as training curves, accuracy vs. sparsity graphs, and visualizations of convolution filters.

Evaluation Metrics. Models should be evaluated using standard classification metrics such as:

- **Test accuracy** on CIFAR-10
- **Training loss and validation accuracy curves**

In addition, students should quantify the **sparsity level** of the trained networks. Let W denote a weight tensor and let $\|W\|_0$ denote the number of nonzero entries. The sparsity level of a layer can be defined as

$$\text{sparsity}(W) = 1 - \frac{\|W\|_0}{|W|},$$

where $|W|$ is the total number of parameters in the layer. Students should report sparsity statistics for different components of the network, for example:

- **Connection sparsity:** the proportion of active weights in each layer.
- **Convolution kernel sparsity:** the proportion of nonzero parameters within convolutional filters.
- **Fully connected layer sparsity:** the proportion of active connections in fully connected layers.

For the methods with dynamic connectivity (SET and RigL), students are encouraged to **track sparsity statistics during training**.

Deliverables

Each team should submit:

- A complete implementation of the sparse training algorithms.
- A report describing the models, methods, and experimental results.
- Quantitative comparisons between static sparse training, SET, and RigL.
- Visualizations illustrating sparsity patterns and learned convolution kernels.